Name: **Chavez, Bliss B**.
Course and Section: IT342 - G2
Project Name: **EbikeRental**
Date: March 9, 2026

Phase 1: Web User Registration and Login

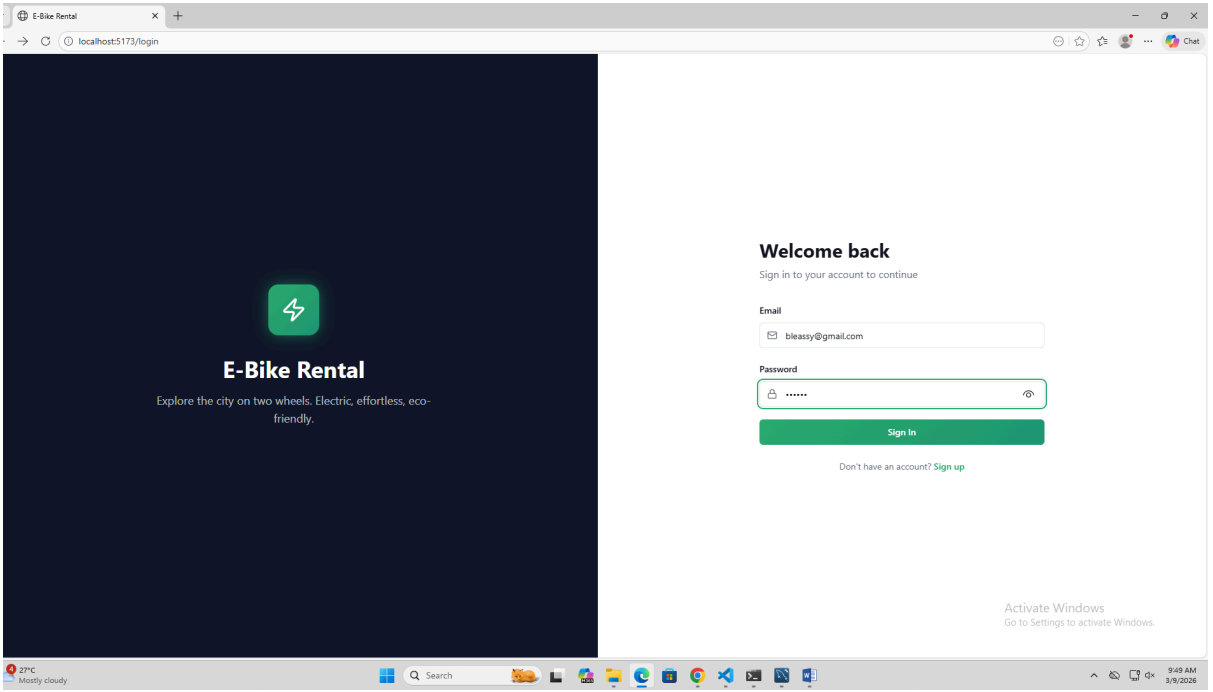
Repository URL	**https://github.com/Bleassy/IT342-Chavez-EbikeRental.git**
Repository Name	**IT342-Chavez-EbikeRental**
Backend Group ID	**edu.cit.chavez**
Backend Artifact ID	**ebikerental**
Base Package	**package edu.cit.chavez.ebikerental;**
Framework	**Spring Boot 3.5.x | Maven | REST API**

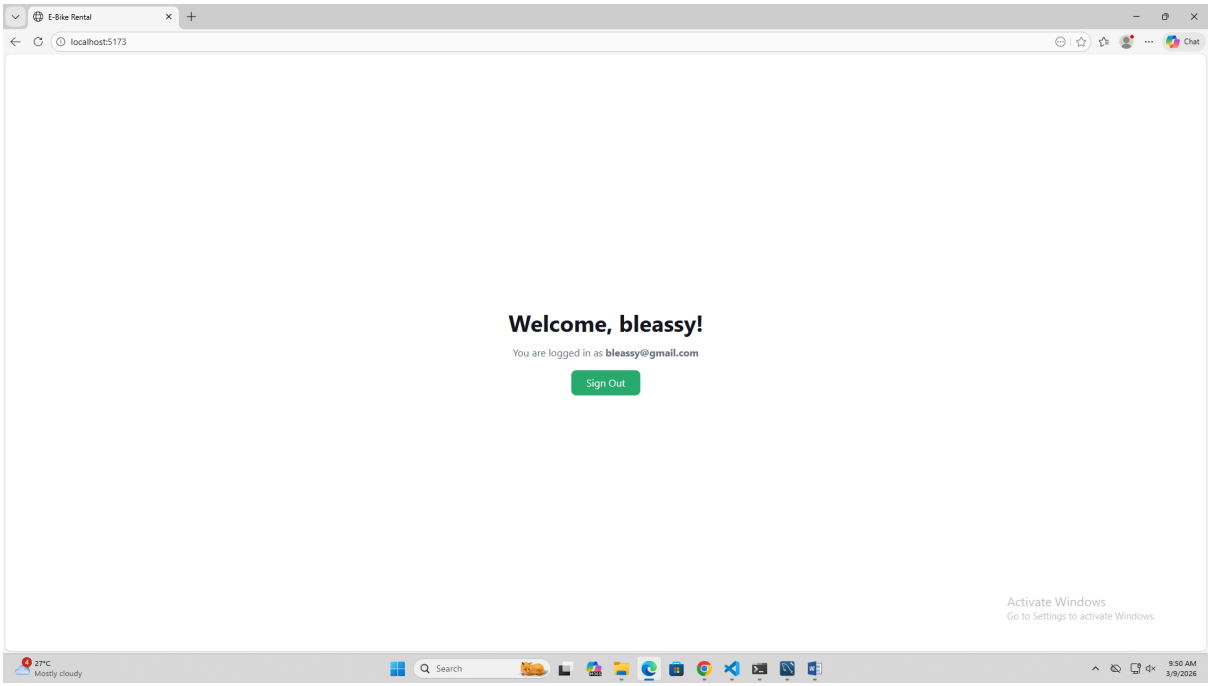
Final Commit for Phase 1

Commit Message:	**IT342 Phase 1 - User Registration and Login Completed**
Commit Hash	**de39188bb8b28de8309ae7a20104a926f906df45**
Commit Link	**https://github.com/Bleassy/IT342-Chavez-EbikeRental/commit/de39188bb8b28de8309ae7a20104a926f906df45**

SCREENSHOT

Login





Successfully registration and login

Headers | Payload | Preview | Response | Initiator | Timing

▼ General

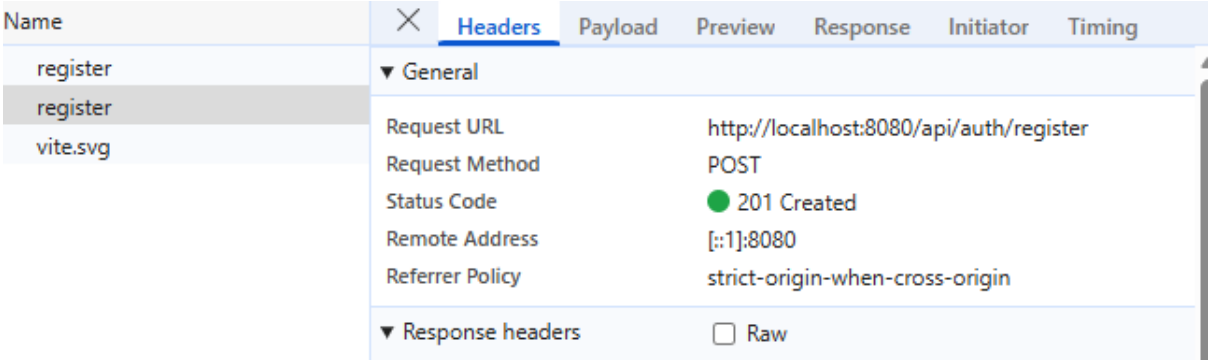
Request URL	http://localhost:8080/api/auth/login
Request Method	POST
Status Code	200 OK
Remote Address	[::1]:8080
Referrer Policy	strict-origin-when-cross-origin

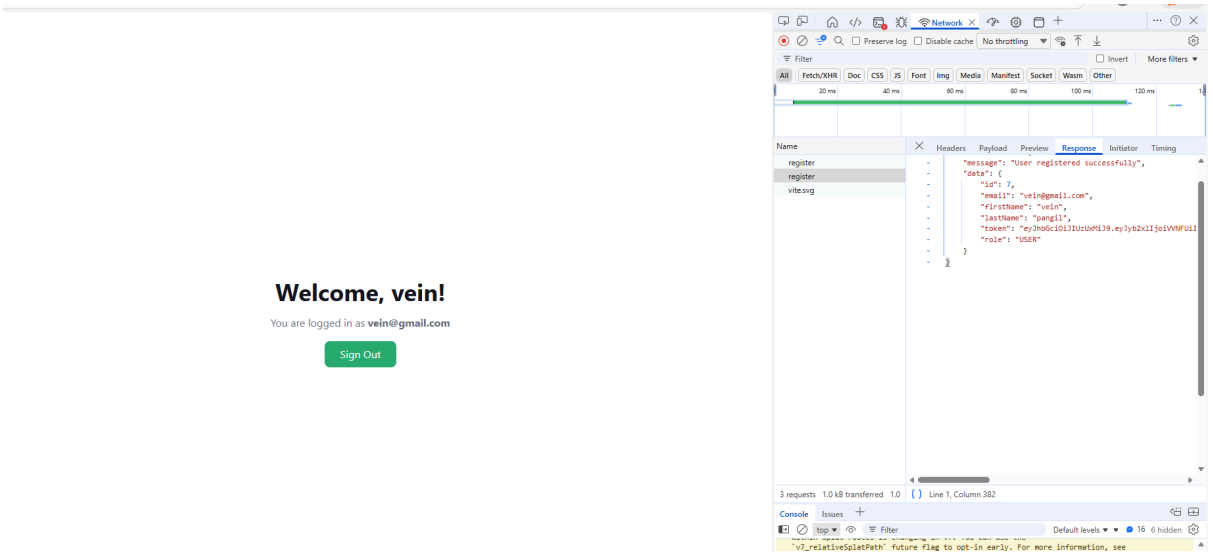
▼ Response headers ☐ Raw

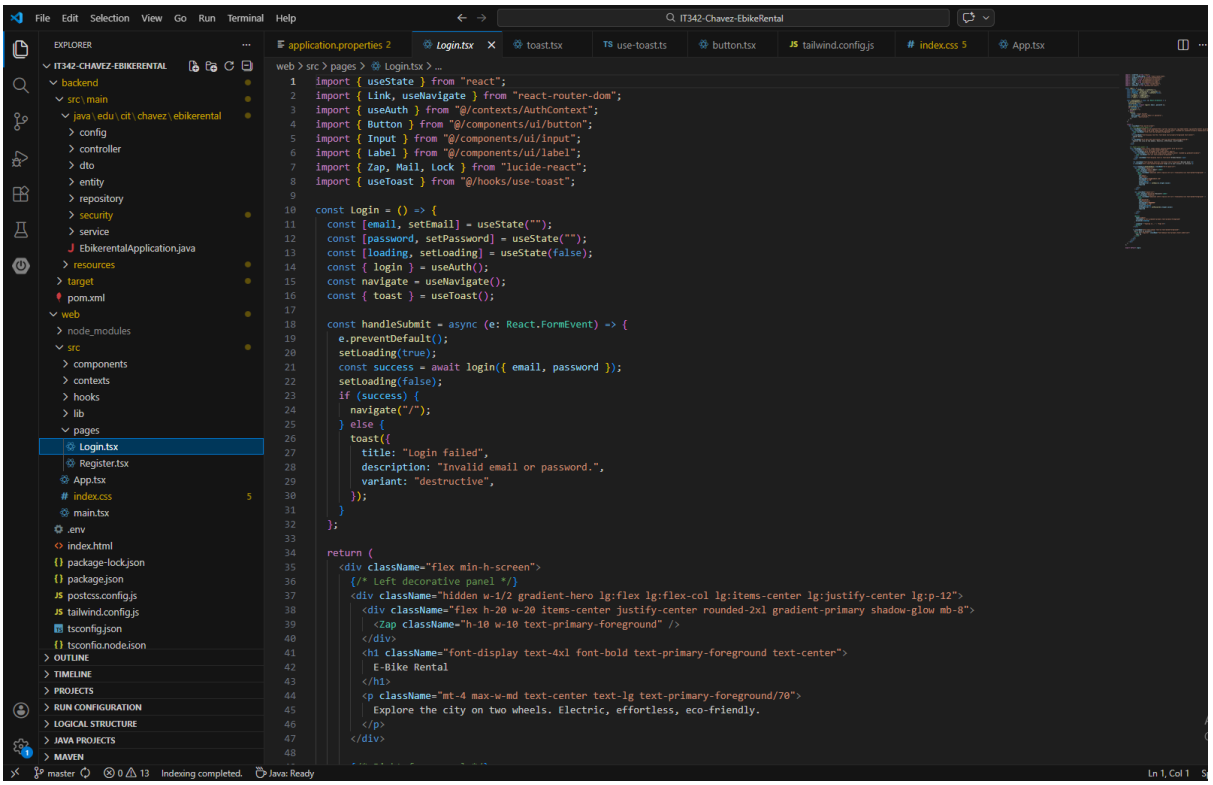
Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	http://localhost:5173
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Connection	keep-alive
Content-Type	application/json
Date	Mon, 09 Mar 2026 01:51:15 GMT
Expires	0
Keep-Alive	timeout=60
Pragma	no-cache
Transfer-Encoding	chunked
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers
X-Content-Type-Options	nosniff

Headers | Payload | Preview | Response | Initiator | Timing


```
{
    "message": "Login successful",
    "data": {
        "id": 6,
        "email": "bleassy@gmail.com",
        "firstName": "bleassy",
        "lastName": "chavez",
        "token": "eyJhbGciOiJIUzUxMiJ9.eyJyb2xlIjoiVVNFUiIsInVzZX
        "role": "USER"
    }
}
```


Name
register
register
vite.svg
Headers
Payload
Preview
Response
Initiator
Timing
General
Request URL
http://localhost:8080/api/auth/register
Request Method
POST
Status Code
201 Created
Remote Address
[::1]:8080
Referrer Policy
strict-origin-when-cross-origin
Response headers
Raw


Welcome, vein!
You are logged in as vein@gmail.com
Sign Out
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
register
register
vite.svg
Headers
Payload
Preview
Response
Initiator
Timing
"message": "User registered successfully",
"data": {
"id": 7,
"email": "vein@gmail.com",
"firstName": "vein",
"lastName": "pangil",
"token": "eyJhbGciOiJIUzUxMiJ9.eyJyb2xlIjoiVVNFUiI
"role": "USER"
}
3 requests 1.0 kB transferred 1.0
Line 1, Column 382
Console
Issues
top
Filter
Default levels
16 6 hidden
`v7_relativeSplatPath` future flag to opt-in early. For more information, see


File
Edit
Selection
View
Go
Run
Terminal
Help
IT342-Chavez-EbikeRental
EXPLORER
IT342-CHAVEZ-EBIKERENTAL
backend
src\main
java\edu\cit\chavez\ebikerental
config
controller
dto
entity
repository
security
service
EbikerentalApplication.java
resources
target
pom.xml
web
node_modules
src
components
contexts
hooks
lib
pages
Login.tsx
Register.tsx
App.tsx
index.css
main.tsx
.env
index.html
package-lock.json
package.json
postcss.config.js
tailwind.config.js
tsconfig.json
tsconfig.node.json
OUTLINE
TIMELINE
PROJECTS
RUN CONFIGURATION
LOGICAL STRUCTURE
JAVA PROJECTS
MAVEN
application.properties 2
Login.tsx
toast.tsx
use-toast.ts
button.tsx
tailwind.config.js
index.css 5
App.tsx
web > src > pages > Login.tsx > ...
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "@/contexts/AuthContext";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Zap, Mail, Lock } from "lucide-react";
import { useToast } from "@/hooks/use-toast";
const Login = () => {
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [loading, setLoading] = useState(false);
const { login } = useAuth();
const navigate = useNavigate();
const { toast } = useToast();
const handleSubmit = async (e: React.FormEvent) => {
e.preventDefault();
setLoading(true);
const success = await login({ email, password });
setLoading(false);
if (success) {
navigate("/");
} else {
toast({
title: "Login failed",
description: "Invalid email or password.",
variant: "destructive",
});
}
};
return (

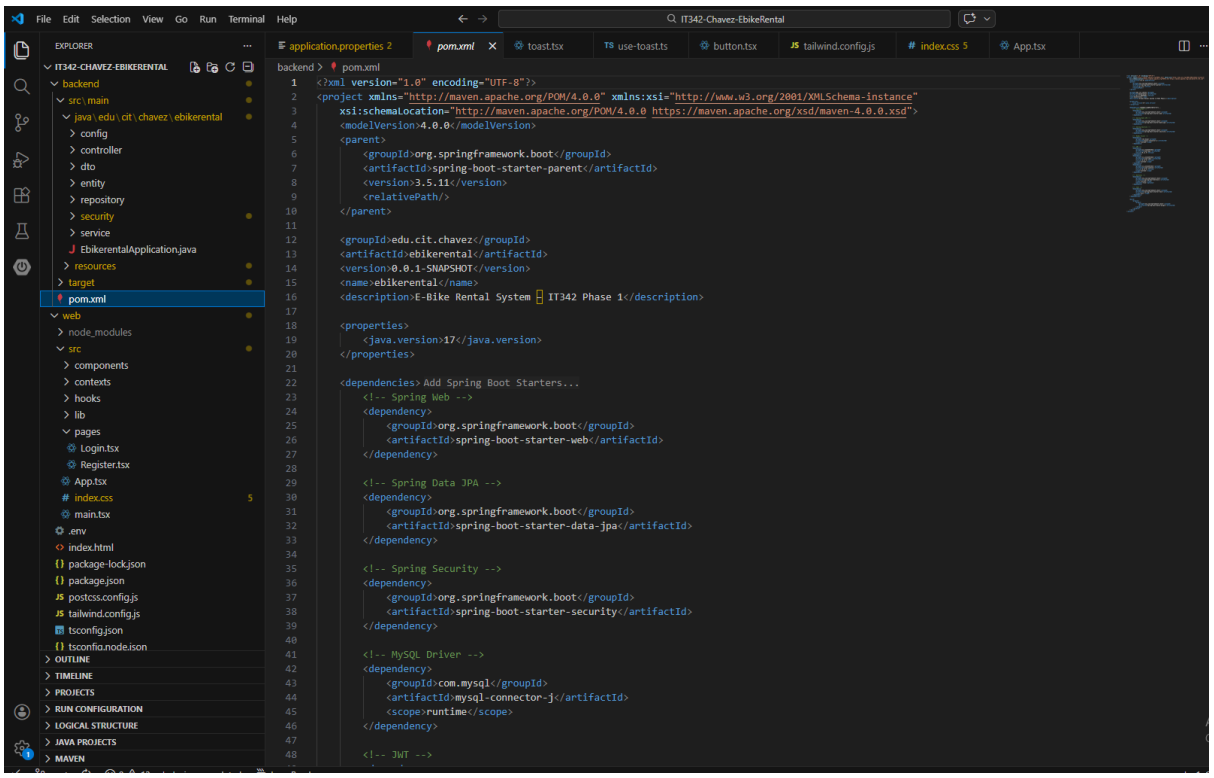
{/* Left decorative panel */}

<Zap className="h-10 w-10 text-primary-foreground" />

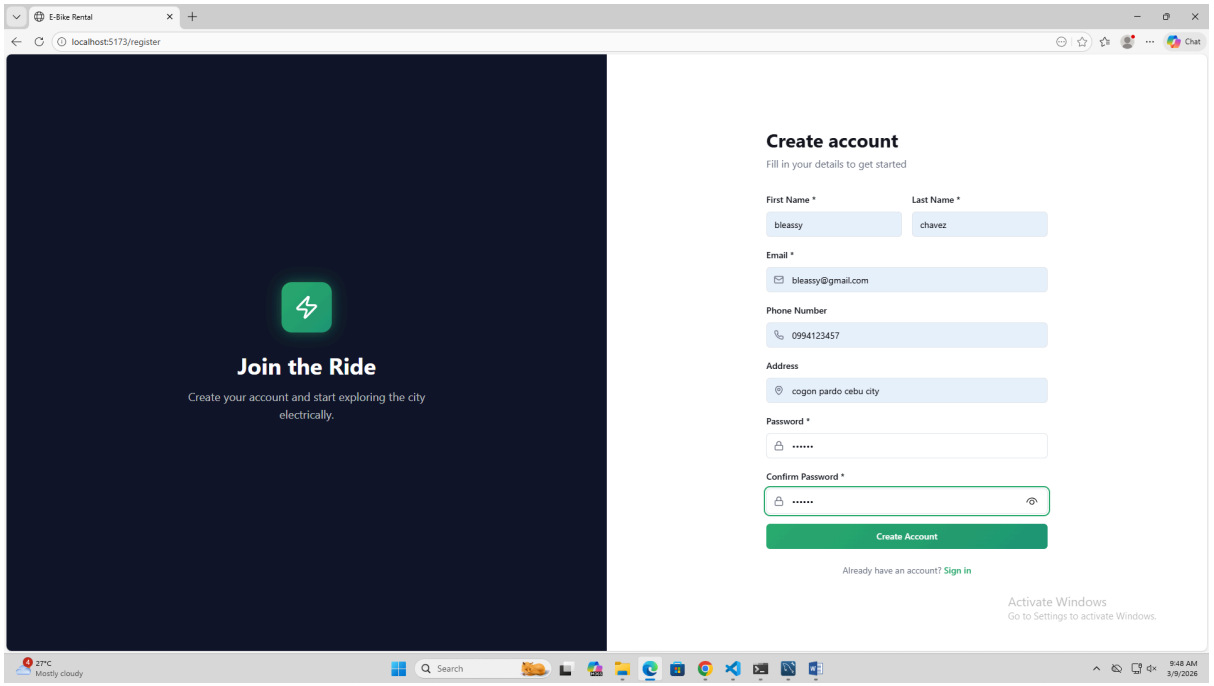
<h1 className="font-display text-4xl font-bold text-primary-foreground text-center">
E-Bike Rental
</h1>

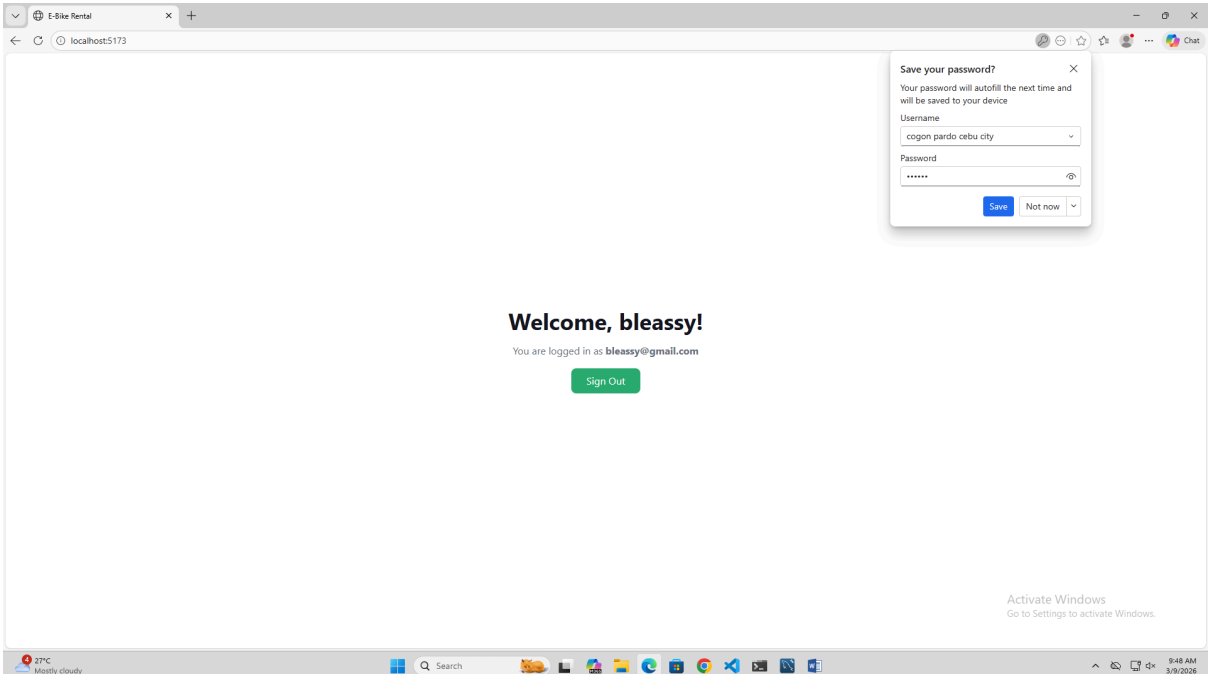
Explore the city on two wheels. Electric, effortless, eco-friendly.

master
0 13
Indexing completed.
Java: Ready
Ln 1, Col 1

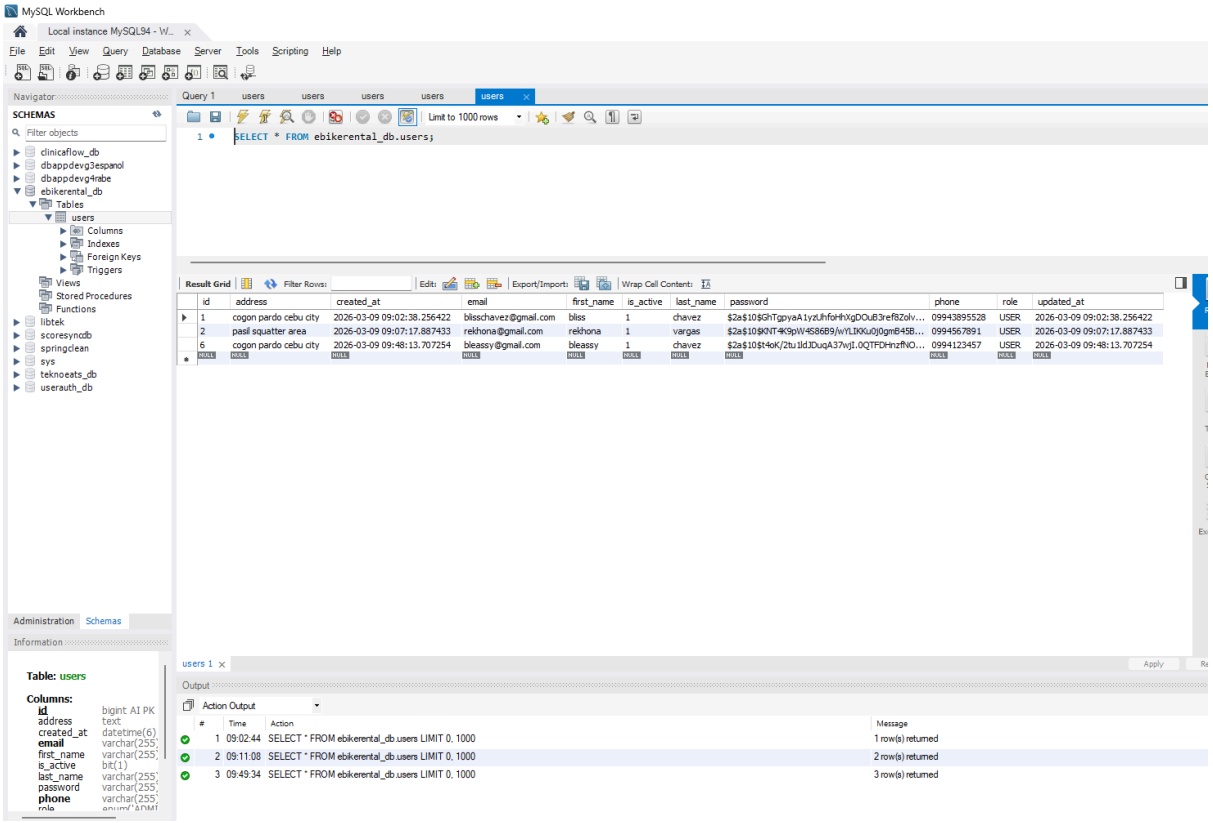


Registration





Database record MYSQL



Short Implementation Summary

E-Bike Rental System IT342 | Phase 1 | Chavez

1. User Registration

Registration Fields Used

The registration form collects the following fields from the user: firstName, lastName, email, password, phone (optional), and address (optional).

Validation Process

Server-side validation is enforced in the *AuthController*. The system checks that firstName, lastName, email, and password are all present and non-blank. If any required field is missing, the API returns HTTP 400 Bad Request with an error message: *"Name, email, and password are required."*

How Duplicate Accounts Are Prevented

Before creating a new user, the system calls *userService.userExists(email)*, which queries the database for any existing record with the same email address. The email column in the *users* table is also declared with a UNIQUE constraint at the database level. If a duplicate is detected, the API returns HTTP 409 Conflict with the message: *"Email already registered."*

Column	Type	Constraints	Description
id	**BIGINT**	**PK, AUTO_INCREMENT**	**Unique user identifier**
first_name	**VARCHAR**	**NOT NULL**	**User's first name**

Passwords are never stored in plaintext. The *UserService* uses Spring Security's BCryptPasswordEncoder to hash the password before saving: *passwordEncoder.encode(request.getPassword())*. BCrypt is a one-way adaptive hashing algorithm that automatically incorporates a salt, making brute-force and rainbow-table attacks infeasible.

2. User Login

Login Credentials Used

Users log in with their registered email address and password.

How the System Verifies Users

The *UserService.loginUser()* method first retrieves the user record from the database by email using *userRepository.findByEmail(email)*. If a matching record is found, it calls *passwordEncoder.matches(rawPassword, storedHash)* to verify the submitted password against the stored BCrypt hash. If the credentials do not match or the email is not found, an empty *Optional* is returned and the controller responds with HTTP 401 Unauthorized.

What Happens After Successful Login

Upon successful verification, the system generates a signed JSON Web Token (JWT) via *JwtTokenProvider.generateToken()*, embedding the user's email, ID, and role. The JWT is returned in the response body alongside the user's basic profile information (id, email, firstName, lastName, role). The client uses this token as a Bearer token in subsequent authenticated API requests.

3. Database Table

Table: user

Column	Type	Constraints	Description

last_name	**VARCHAR**	**NOT NULL**	**User's last name**
email	**VARCHAR**	**NOT NULL, UNIQUE**	**Login email address**
password	**VARCHAR**	**NOT NULL**	**BCrypt-hashed password**
phone	**VARCHAR**	**UNIQUE, nullable**	**Optional contact number**
address	**TEXT**	**nullable**	**Optional home address**
role	**ENUM**	**NOT NULL**	**USER or ADMIN**
is_active	**BOOLEAN**	**NOT NULL, default true**	**Account active flag**
created_at	**DATETIME**	**NOT NULL, immutable**	**Record creation timestamp**
updated_at	**DATETIME**	**NOT NULL**	**Last update timestamp**

4. API Endpoints

Method	Endpoint	Description	Response
POST	**/auth/register**	**Register a new user account**	**201 Created + JWT token**
POST	**/auth/login**	**Authenticate an existing user**	**200 OK + JWT token**

Both endpoints are mapped under the base path **/auth** and accept/return JSON. On success, both return an *AuthResponse* object containing the user ID, name, email, role, and a signed JWT bearer token used for authenticating subsequent requests.